ASSIGNMENT - 8.2

2303A51091
Batch : 29

Task Description
Task 1 – Test-Driven Development for Even/Odd Number Validator
• Use AI tools to first generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests.
Requirements:
• Input must be an integer
• Handle zero, negative numbers, and large integers
Example Test Scenarios:
is_even(2) → True
is_even(7) → False
is_even(0) → True
is_even(-4) → True
is_even(9) → False
Expected Output -1
• A correctly implemented is_even() function that passes all AI-generated test cases

task1.py > ...

```python
# ------------------------------
# Test-Driven Development (TDD)
# Even/Odd Number Validator
# ------------------------------


# Step 1: Define test cases
def test_is_even():
    # Positive numbers
    assert is_even(2) == True
    assert is_even(7) == False

    # Zero
    assert is_even(0) == True

    # Negative numbers
    assert is_even(-4) == True
    assert is_even(-9) == False

    # Large integers
    assert is_even(10**15) == True
    assert is_even(10**15 + 1) == False

    # Invalid input (should raise TypeError)
    try:
        is_even("10")
    except TypeError:
        pass
    else:
        assert False, "TypeError not raised for string input"


# Step 2: Implement function
def is_even(n):
    if not isinstance(n, int):
        raise TypeError("Input must be an integer")
    return n % 2 == 0


# Step 3: Run tests
if __name__ == "__main__":
    test_is_even()
    print("All test cases passed successfully!")

```

Ln 44, Col 1 Spa

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment - 8.2.py/task1.py"
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assi
stent.py/assignment - 8.2.py/task1.py"
All test cases passed successfully!
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py %
```

Observation:

The Test-Driven Development (TDD) approach ensures that test cases are written before implementing the function, improving code reliability.

Writing assert-based test cases helped verify the correctness of the `is_even(n)` function for positive, negative, zero, and large integer inputs.

Input validation using `isinstance()` ensures that only integer values are accepted, and a `TypeError` is raised for invalid inputs.

The modulus operator (`n % 2 == 0`) correctly determines even numbers, including negative integers.

Running all test cases successfully confirms that the function satisfies functional requirements and handles edge cases properly.

Task Description

Task 2 – Test-Driven Development for String Case Converter

- **Ask AI to generate test cases for two functions:**
- **to_uppercase(text)**
- **to_lowercase(text)**

Requirements:

- **Handle empty strings**
- **Handle mixed-case input**
- **Handle invalid inputs such as numbers or None**

Example Test Scenarios:

to_uppercase("ai coding") → "AI CODING"

to_lowercase("TEST") → "test"

to_uppercase("") → ""

to_lowercase(None) → Error or safe handling

Expected Output -2

- **Two string conversion functions that pass all AI-generated test cases with safe input handling.**

task2.py > ...

```python
def test_string_case_functions():
        else:
            assert False, "TypeError not raised for integer input"

    try:
        to_lowercase(None)
    except TypeError:
        pass
    else:
        assert False, "TypeError not raised for None input"


# Step 2: Implement Functions
def to_uppercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.upper()


def to_lowercase(text):
    if not isinstance(text, str):
        raise TypeError("Input must be a string")
    return text.lower()


# Step 3: Run Tests
if __name__ == "__main__":
    test_string_case_functions()
    print("All test cases passed successfully!")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment - 8.2.py/task2.py"
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assi
stent.py/assignment - 8.2.py/task2.py"
All test cases passed successfully!
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py %
```

Observation:

1. The Test-Driven Development (TDD) approach ensures that test cases are written before implementing the string conversion functions.

2. The `.upper()` and `.lower()` built-in string methods correctly convert text while preserving non-alphabet characters.

3. Edge cases such as empty strings are handled safely without errors.

4. Input validation using `isinstance(text, str)` ensures that invalid inputs like integers or `None` raise a `TypeError`.

5. Successful execution of all test cases confirms that both functions meet functional and safety requirements.

Task Description
Task 3 – Test-Driven Development for List Sum Calculator
• Use AI to generate test cases for a function sum_list(numbers)
that calculates the sum of list elements.
Requirements:
• Handle empty lists

• Handle negative numbers
• Ignore or safely handle non-numeric values
Example Test Scenarios:
sum_list([1, 2, 3]) → 6
sum_list([]) → 0
sum_list([-1, 5, -4]) → 0
sum_list([2, "a", 3]) → 5
Expected Output 3
• A robust list-sum function validated using AI-generated test cases.

task3.py > ...

```python
# ----------------------------------------
# Test-Driven Development (TDD)
# List Sum Calculator
# ----------------------------------------


# Step 1: Define Test Cases
def test_sum_list():

    # Normal case
    assert sum_list([1, 2, 3]) == 6

    # Empty list
    assert sum_list([]) == 0

    # Negative numbers
    assert sum_list([-1, 5, -4]) == 0

    # Mixed numeric and non-numeric values
    assert sum_list([2, "a", 3]) == 5

    # Large numbers
    assert sum_list([10**10, 10**10]) == 2 * 10**10

    # List with only non-numeric values
    assert sum_list(["a", None, "b"]) == 0


# Step 2: Implement Function
def sum_list(numbers):

    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment - 8.2.py/task3.py"
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assi
stent.py/assignment - 8.2.py/task3.py"
All test cases passed successfully!
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py %
```

Observation:

1. Test-Driven Development (TDD) ensures that test cases are written before implementing the `sum_list()` function.
2. The function correctly handles empty lists by returning `0`, preventing runtime errors.
3. Negative numbers are properly included in the summation logic.
4. Non-numeric values such as strings and `None` are safely ignored, improving robustness.
5. Input validation ensures that only list inputs are accepted, raising a `TypeError` for invalid input types.

Task Description
Task 4 – Test Cases for Student Result Class
• Generate test cases for a StudentResult class with the following methods:
• add_marks(mark)
• calculate_average()
• get_result()
Requirements:
• Marks must be between 0 and 100
• Average ≥ 40 → Pass, otherwise Fail

Example Test Scenarios:
Marks: [60, 70, 80] → Average: 70 → Result: Pass
Marks: [30, 35, 40] → Average: 35 → Result: Fail
Marks: [-10] → Error
Expected Output -4
• A fully functional StudentResult class that passes all AI-generated test

task4.py > ...

```python
# ----------------------------------------
# Test-Driven Development (TDD)
# Student Result Class
# ----------------------------------------


# Step 1: Define Test Cases
def test_student_result():

    # Test Case 1: Pass scenario
    student1 = StudentResult()
    student1.add_marks(60)
    student1.add_marks(70)
    student1.add_marks(80)

    assert student1.calculate_average() == 70
    assert student1.get_result() == "Pass"

    # Test Case 2: Fail scenario
    student2 = StudentResult()
    student2.add_marks(30)
    student2.add_marks(35)
    student2.add_marks(40)

    assert student2.calculate_average() == 35
    assert student2.get_result() == "Fail"

    # Test Case 3: Invalid negative marks
    student3 = StudentResult()
    try:
        student3.add_marks(-10)
    except ValueError:
        pass
    else:
        assert False, "ValueError not raised for negative marks"


# Step 2: Implement StudentResult class
class StudentResult:
    def __init__(self):
        self._marks = []

    def add_marks(self, mark):
        if not isinstance(mark, (int, float)):
            raise TypeError("Mark must be a number")
```

Ln 64, Col 1

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment - 8.2.py/task4.py"
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py % /usr/bin/python3 "/Users/arepally
ramcharan/ai assistent.py/assignment - 8.2.py/task4.py"
All test cases passed successfully!
arepallyramcharan@Arepallys-MacBook-Air assignment - 8.2.py %
```

Observation:

1. The Test-Driven Development (TDD) approach ensures all class methods are validated before final implementation.
2. The `add_marks()` method enforces strict validation, allowing only marks between 0 and 100.
3. The `calculate_average()` method safely handles empty mark lists by returning 0.
4. The `get_result()` method correctly determines Pass or Fail based on the average threshold of 40.
5. Exception handling improves robustness by preventing invalid data from being stored.

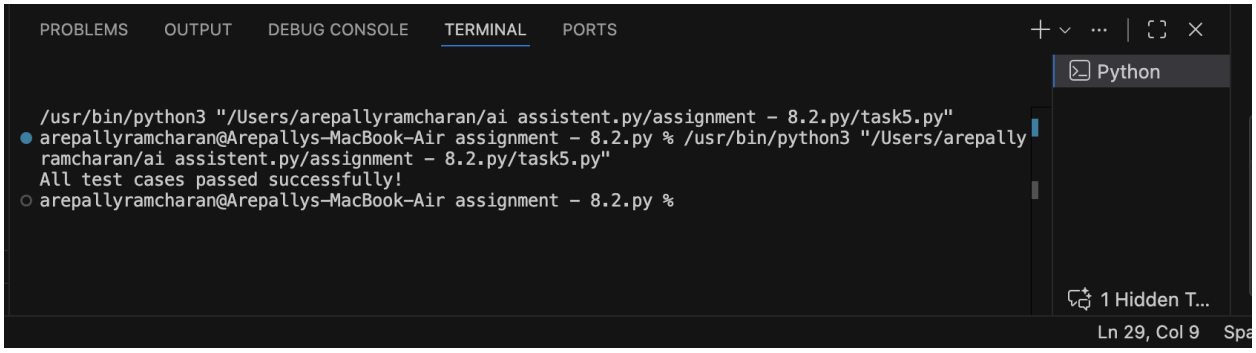
Task Description
Task 5 – Test-Driven Development for Username Validator
Requirements:
• Minimum length: 5 characters
• No spaces allowed
• Only alphanumeric characters

Example Test Scenarios:
is_valid_username("user01") → True
is_valid_username("ai") → False
is_valid_username("user name") → False
is_valid_username("user@123") → False
Expected Output 5
A username validation function that passes all AI-generated test cases.

task5.py > ...

```python
# ----------------------------------------
# Test-Driven Development (TDD)
# Username Validator
# ----------------------------------------


# Step 1: Define Test Cases
def test_is_valid_username():

    # Valid username
    assert is_valid_username("user01") == True

    # Minimum length check
    assert is_valid_username("ai") == False
    assert is_valid_username("user1") == True  # exactly 5 characters

    # No spaces allowed
    assert is_valid_username("user name") == False

    # Only alphanumeric characters
    assert is_valid_username("user@123") == False
    assert is_valid_username("user_123") == False

    # Edge cases
    assert is_valid_username("") == False
    assert is_valid_username("12345") == True

    # Invalid input type
    try:
        is_valid_username(None)
    except TypeError:
        pass
    else:
```

Output:



Observation:

- Test-Driven Development (TDD) ensures validation rules are clearly defined before implementing the username validator.
- The function enforces a minimum length of 5 characters to maintain basic security standards.
- The `.isalnum()` method ensures only alphanumeric characters are allowed.
- Space detection prevents invalid usernames with whitespace.
- Type checking improves robustness by raising `TypeError` for non-string inputs.